GUÍA PEDAGÓGICA Y TÉCNICA PASO A PASO

SISTEMA FCT

Manual de Comprensión Integral del Proyecto y Guía de Trabajo Colaborativo en Git

Orientado a: Estudiantes y Desarrolladores Junior

Enfoque: Arquitectura MVC, Lógica de Negocio, Flujo de Datos y Trabajo en Equipo

Estado del Sistema: PHP 8.2 Nativo / PDO / MySQL - MariaDB

1. Filosofía General del Proyecto y Contexto

El **Sistema FCT** (Formación en Centros de Trabajo) es una aplicación web desarrollada en PHP 8.2 nativo orientada a coordinar la relación entre un centro educativo, las empresas colaboradoras y el alumnado. Su propósito funcional es gestionar el ciclo completo de las prácticas profesionales: desde el registro de usuarios y empresas hasta la firma de convenios, publicación de ofertas, disponibilidad de alumnos y asignación de plazas.

> 💡 **¿Por qué se siente complejo?**
>
> El proyecto no utiliza frameworks comerciales como Laravel o Symfony, ni librerías de terceros instaladas por Composer o paquetes de npm. Aunque esto reduce el número de archivos externos, significa que toda la infraestructura (enrutamiento, seguridad, conexión a base de datos y renderizado) fue escrita de forma artesanal. Comprender el proyecto requiere entender cómo funcionan estas piezas de infraestructura desde cero.

Roles del Sistema

Rol	Descripción y Uso Previsto	Permisos Clave
Administrador	Gestión global del sistema y cuentas internas.	Creación de usuarios del personal, activar/desactivar cuentas, revisión de convenios/ofertas y asignaciones.
Coordinador	Gestión operativa de prácticas del centro.	Revisar convenios, aprobar/rechazar ofertas, realizar matching de alumnos y crear asignaciones.
Observador	Perfil de consulta e inspección sin capacidad de edición.	Visualizar asignaciones, consultas globales y descargar PDFs de convenios.
Empresa	Entidad externa que ofrece plazas de prácticas.	Registrar sedes, subir convenios en PDF y publicar ofertas de prácticas.
Alumno	Estudiante candidato a realizar la FCT.	Registrar su ventana de disponibilidad (fechas y turno) y consultar sus asignaciones.

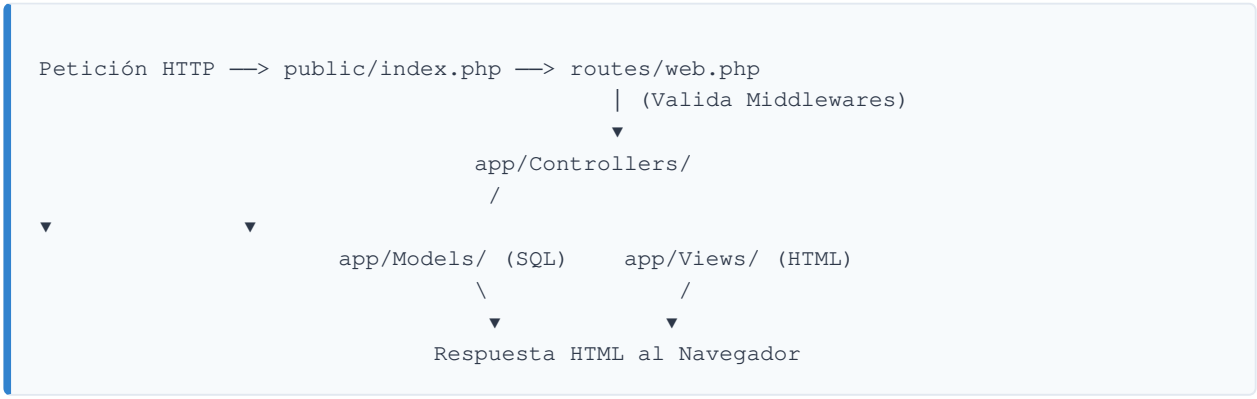
2. Arquitectura Técnica Explicada Paso a Paso

La aplicación sigue el patrón de diseño **MVC (Modelo-Vista-Controlador)** combinado con el patrón **Front Controller** (Punto de Entrada Único).

El Viaje de una Petición (Request Cycle)

Cuando un usuario interactúa con la aplicación (por ejemplo, al hacer clic en "Iniciar Sesión"), la petición recorre exactamente los siguientes pasos:

1. **Punto de Entrada (`public/index.php`):** El servidor web Apache o PHP dirige todas las solicitudes a este archivo único. Se inicia la sesión PHP (`session_start()`), se configuran las cabeceras de seguridad HTTP y se cargan las clases necesarias.
2. **Lookup de Ruta (`routes/web.php`):** El Front Controller lee el parámetro `?action=...` de la URL. Consulta la matriz de rutas para verificar si la acción existe, qué métodos HTTP permite (GET o POST) y qué roles tienen autorización.
3. **Validación por Middlewares:**
   - `AuthMiddleware`: Si la ruta requiere estar autenticado, comprueba que exista una sesión válida y que la cuenta del usuario no esté desactivada en la base de datos.
   - `CsrfMiddleware`: Si la petición es de tipo POST (envío de formulario), verifica que el token CSRF coincida para proteger contra ataques de falsificación de petición.
4. **Ejecución del Controlador (`app/Controllers/`):** Se instancia la clase correspondiente (ej. `AuthController`) y se invoca el método asignado. El controlador lee y valida los datos recibidos.
5. **Consulta al Modelo (`app/Models/`):** Si se necesitan o modifican datos, el controlador llama al Modelo correspondiente. El modelo ejecuta las consultas SQL mediante PDO y devuelve arrays de datos o confirmaciones.
6. **Renderizado de la Vista (`app/Views/`):** El controlador pasa los datos a `View::render()`. La vista genera el código HTML, que se captura y se inyecta dentro de la plantilla común (`app/Views/layouts/main.php`) para enviarse al navegador del usuario.



3. Módulos del Sistema y Reglas de Negocio

3.1 Registro y Seguridad de Usuarios

Las contraseñas se almacenan de forma segura utilizando el algoritmo **BCrypt** (vía `password_hash()`) y se verifican mediante `password_verify()`. Los formularios exigen contraseñas de entre 12 y 72 caracteres.

3.2 Registro de Alumnado

Al registrar un alumno, el sistema valida la fecha de nacimiento para verificar que tenga al menos **16 años**. Si el alumno es menor de 18 años, el formulario exige obligatoriamente los datos del tutor legal (nombre, DNI, SS, teléfono, dirección y correo). Todo el alta (usuario, expediente, dirección y tutor) se realiza en una **transacción SQL**; si un dato falla, se revierte todo.

3.3 Convenios y Subida Segura de Archivos

Las empresas suben un convenio en formato PDF (máximo 10 MB). Por seguridad, el archivo no se guarda en una carpeta pública, sino en `storage/uploads/convenios/`. Se le asigna un nombre aleatorio único. Para descargarlo, se pasa por un controlador que verifica que la empresa sea la dueña del convenio o que el usuario sea personal autorizado del centro.

3.4 Ofertas de Prácticas

Una oferta debe asociarse a un ciclo formativo existente, indicar turno (Mañana, Tarde e Indiferente) y número de plazas (entre 1 y 100). Para que una empresa pueda publicar una oferta, debe existir un **convenio aprobado** que cubra completamente el rango de fechas de la oferta. La oferta nace en estado `Pendiente` y debe ser aprobada por un Administrador o Coordinador.

3.5 Algoritmo de Matching (Emparejamiento de Alumnos)

Para proponer candidatos a una oferta, el sistema realiza un filtro multinivel:

- Misma especialidad/ciclo entre la oferta y el alumno.
- Ventana de disponibilidad del alumno que cubra **completamente** el rango de fechas de la oferta.
- Incompatibilidad de turnos resuelta (mismo turno o turno 'Indiferente').
- El alumno no debe tener otra asignación actualmente en estado `Asignada`.

3.6 Asignación de Plazas y Control de Concurrencia

Cuando un Coordinador asigna un alumno a una oferta, el sistema bloquea los registros de la base de datos mediante transacciones con la cláusula `FOR UPDATE`. Esto garantiza que si dos coordinadores intentan hacer una asignación al mismo segundo, no se sobrepase el límite de plazas ofertadas.

4. Guía de Lectura Estratégica del Código

Para no abrumarte intentando leer todos los archivos a la vez, sigue esta ruta pedagógica de lectura "Vertical" (de principio a fin por funcionalidad):

> 📌 **Secuencia de Aprendizaje Sugerida**
>
> 1. **Día 1 - Núcleo e Infraestructura:** Examina `public/index.php`, `routes/web.php`, `app/Core/View.php` y `app/Middleware/CsrfMiddleware.php`.
> 2. **Día 2 - Flujo Simple (Login):** Sigue el camino de `AuthController.php` ➔ `Usuario.php` ➔ `app/Views/auth/login.php`.
> 3. **Día 3 - Flujo de Formularios Complejos:** Analiza `AlumnoController.php` y `ConvenioController.php` para entender las transacciones y subida de archivos.
> 4. **Día 4 - Lógica Compleja de Negocio:** Revisa `OfertaController.php`, `AsignacionController.php` y los modelos `Oferta.php` y `Asignacion.php`.

5. Guía Práctica de Trabajo Colaborativo en Git

En un equipo de desarrollo, nunca se debe trabajar directamente sobre la rama principal (`main`). A continuación se detallan las órdenes y el flujo de trabajo paso a paso:

5.1 PreparaciónDiaria

Antes de escribir una sola línea de código, actualiza tu repositorio y crea una rama para tu tarea:

```
# 1. Cambiar a la rama principal
git checkout main

# 2. Descargar los cambios hechos por tus compañeros
git pull origin main

# 3. Crear y cambiar a tu propia rama de trabajo
git checkout -b feature/pantalla-mis-asignaciones
```

5.2 Guardar Cambios (Commits)

Haz guardados frecuentes de tareas pequeñas y bien definidas:

```
# Comprobar qué archivos has modificado
git status

# Ver los cambios exactos dentro de un archivo
git diff app/Controllers/AsignacionController.php

# Agregar únicamente los archivos que deseas guardar
git add app/Controllers/AsignacionController.php

# Crear el commit con un mensaje descriptivo
git commit -m "Añadida comprobación de estado al listar mis asignaciones"
```

5.3 Subir el Trabajo y Solicitar Revisión (Pull Request)

```
# Subir tu rama al servidor remoto (GitHub/GitLab)
git push origin feature/pantalla-mis-asignaciones
```

Posteriormente, entra en la plataforma web (GitHub) y crea un **Pull Request (PR)** para pedir que tus compañeros revisen tu código antes de unirlo a `main`.

5.4 Resolución de Conflictos

Si dos personas editaron el mismo archivo en las mismas líneas, Git solicitará resolver el conflicto:

```
# 1. En tu rama, trae los cambios de main
git merge main

# 2. Abre VS Code. Verás marcas como:
# <<<<<<< HEAD (Tu código)
# =======
# >>>>>>> main (Código del compañero)

# 3. Borra las líneas innecesarias, guarda, haz git add y confirma el merge:
git add .
git commit -m "Resuelto conflicto en el controlador de asignaciones"
```

6. Glosario de Conceptos Clave

Término	Explicación Sencilla
PDO	Capa de abstracción de PHP para conectar con la base de datos de forma segura mediante consultas preparadas.
CSRF Token	Código secreto temporal incluido en formularios para verificar que el envío proviene realmente del usuario legítimo.
Transacción SQL	Grupo de órdenes a la base de datos que se ejecutan todas con éxito (COMMIT) o se cancelan todas (ROLLBACK) si hay un error.
FOR UPDATE	Cláusula SQL que bloquea temporalmente una fila para impedir que otros usuarios la modifiquen al mismo tiempo.
BCrypt	Algoritmo de encriptación no reversible utilizado para guardar contraseñas de forma segura en la base de datos.